

EduMatric — Explicação Muito Simples

Este documento explica o projecto **sem termos difíceis**.
Se o `README.md` é o manual técnico, este ficheiro é a explicação “para quem está a começar do zero”.

Índice rápido

- [O que é](#)
- [Quem usa](#)
- [Fluxo do sistema](#)
- [Instalar e rodar \(do zero\)](#) ← para os colegas leigos
- [Logins de teste](#)
- [Problemas comuns](#)

1. O que é o EduMatric?

Imagina um **site da escola** onde:

- o **aluno** pede para se matricular;
- a **secretaria** vê quem pediu e decide aprovar ou recusar;
- o **director** vê números gerais e pode **abrir ou fechar** as matrículas.

Não é uma app no telemóvel. É um **site que corre no computador**, no endereço:

<http://localhost:3000>

("localhost" = o teu próprio computador, quando o servidor está ligado.)

2. O que o sistema faz, em uma frase?

Organiza pedidos de matrícula — quem pediu, se está à espera, se foi aceite ou recusado.

3. Quem usa o site?

Pessoa	O que faz
Aluno	Preenche o formulário de matrícula, consulta o estado, e depois de aprovado faz login
Secretaria	Vê a lista de alunos, aprova, recusa, trata reconfirmações
Director	Vê totais (quantos pendentes, aprovados, etc.) e abre/fecha matrículas

Não há registo público de “criar conta”.
O aluno **não escolhe senha** no início — só envia o pedido.
A secretaria e o director **têm utilizador e senha** fixos na base de dados.

4. História do fluxo (do início ao fim)

Passo A — Aluno quer matricular-se

1. Entra no site → **Fazer matrícula**
2. Preenche nome, email, curso, telefone, etc.
3. Pode enviar **foto e certificado** (ficheiros)
4. Clica em enviar

O que acontece por trás: os dados vão para a **base de dados**, na tabela `alunos`, com estado **Pendente**.

Passo B — Aluno quer saber “já fui aceite?”

1. Vai a **Consultar estado da matrícula**
2. Mete **nome + email** (os mesmos do formulário)
3. O site mostra: Pendente, Aprovado ou Recusado

Se foi **recusado**, pode aparecer o motivo.

Passo C — Secretaria decide

1. Faz login em **Secretaria**
2. Vê a lista de alunos
3. Para cada um **Pendente** pode:
 - **Aprovar** → o sistema cria um **ID oficial** (ex: `JOAB1001`)
 - **Recusar** → escreve um motivo

Importante: só depois de **aprovado** é que o aluno tem ID para login.

Passo D — Aluno aprovado entra na sua área

1. Login do aluno: **ID oficial** + **email**
2. Vê o painel com os seus dados

Passo E — Director controla o “portão”

1. Faz login em **Director**
2. Vê quantos alunos existem (total, pendentes, aprovados, recusados)
3. Pode **abrir** ou **fechar** matrículas para novos pedidos

Quando está **fechado**, novos alunos não conseguem (ou não deviam) submeter matrícula — conforme a lógica do sistema.

5. Estados de uma matrícula (linguagem simples)

Estado	Significado
Pendente	Pedido enviado, secretaria ainda não decidiu
Aprovado	Aceite; aluno recebe ID oficial
Recusado	Não aceite; pode haver motivo guardado

6. O que é cada “peça” do projecto?

Pensa no projecto como uma **loja**:

Peça	Analogia	Ficheiro / pasta
Porta da loja	Onde tudo começa	<code>server.js</code>
Menu / caminhos	“Se fores a este URL, faz isto”	<code>src/routes/index.js</code>
Funcionários	Recebem o pedido e decidem o que fazer	<code>src/controllers/</code>
Armazém	Onde os dados ficam guardados (PostgreSQL)	base de dados <code>edumatic</code>
Quem vai buscar dados ao armazém	Fala com a base de dados	<code>src/models/</code>
Ecrãs que o utilizador vê	Páginas HTML bonitas	<code>src/views/</code> (ficheiros <code>.ejs</code>)
Imagens e CSS	Aspecto visual	<code>public/</code>
Fotos e PDFs dos alunos	Pasta de uploads	<code>public/uploads/</code>

Base de dados (PostgreSQL)

É como uma **folha Excel gigante** no computador, mas mais segura e rápida.

Tabelas principais:

- **alunos** — todos os candidatos
- **secretaria** — conta da secretaria
- **diretor** — conta do director
- **configuracoes** — opções do sistema (ex: matrículas abertas sim/não)

Ligação configurada em: `src/config/db.js`

```
Computador → PostgreSQL → base "edumatic"
Utilizador: postgres
Senha: 1234
```

7. Guia completo: instalar, clonar e rodar

Esta secção é para **qualquer colega que nunca programou** mas precisa de ter o site a funcionar no computador.

Checklist — o que precisas no PC

#	Programa	Para quê?	Versão mínima
1	Git	Descarregar (clonar) o código do projecto	Qualquer recente
2	Node.js	Correr o site (servidor)	18 ou superior (recomendado 20+)
3	PostgreSQL	Guardar alunos, senhas, etc.	14 ou superior
4	Navegador	Chrome, Firefox ou Edge	Qualquer
5	Terminal	Onde escreves os comandos	Já vem no sistema

Espaço em disco: cerca de 500 MB–1 GB (programas + projecto).

Internet: só é obrigatória para **clonar** o repositório e para `npm install` (descarrega pacotes). Depois o site pode correr **offline** no teu PC.

O que é cada coisa? (2 minutos de leitura)

- **Terminal / consola** — janela preta ou aplicativo onde escreves comandos de texto (não clicas em botões do site).
- **Git** — ferramenta para copiar o projecto do GitHub (ou outro sítio) para o teu computador.
- **Clonar** — `git clone` = “faz uma cópia completa da pasta do projecto”.
- **Node.js** — motor que executa o JavaScript do servidor (`server.js`).
- **npm** — vem com o Node; instala bibliotecas do projecto (`npm install`).
- **PostgreSQL** — programa que guarda os dados numa **base de dados** chamada `edumatic`.

Passo 0 — Instalar Git, Node.js e PostgreSQL

Faz isto **uma vez** por computador.

Git

- **Windows:** <https://git-scm.com/download/win> — instala com “Next, Next” e deixa as opções por defeito.
- **Linux (Debian/Ubuntu/Kali):** `sudo apt update && sudo apt install -y git`
- **macOS:** `xcode-select --install` ou instala Git do site acima.

Confirma no terminal:

```
git --version
```

Deve aparecer algo como `git version 2.x.x`.

Node.js

- Site oficial: <https://nodejs.org/> — escolhe a versão **LTS** (recomendada).

- Instala normalmente. O **npm** vem junto.

Confirma:

```
node -v
npm -v
```

Exemplo: `v20.x.x` e `10.x.x`.

PostgreSQL

- Site: <https://www.postgresql.org/download/>
- Na instalação, o instalador pede uma **senha para o utilizador postgres**.
Anota essa senha. O projecto está configurado com senha **1234** em `src/config/db.js`.
 - Se na instalação usares outra senha, tens de **alterar** `src/config/db.js` para a tua senha, **ou** mudar a senha do postgres no PostgreSQL para **1234**.

Confirma que o serviço está a correr:

- **Windows:** serviço "postgresql" no Gestor de Tarefas / Serviços.
- **Linux:** `sudo systemctl start postgresql` (e `enable` para arrancar com o PC).

Teste rápido (pede a senha do postgres):

```
psql -U postgres -c "SELECT 1;"
```

Se der erro de ligação, o PostgreSQL não está ligado.

Passo 1 — Clonar o projecto (copiar do GitHub)

Abre o **terminal** e vai para a pasta onde queres o projecto (ex: Ambiente de trabalho):

```
cd ~/Desktop
```

URL do repositório (exemplo — usa o link que o teu colega te passar):

```
https://github.com/afonsozango/PROJECTO-WEB.git
```

Comando para clonar:

```
git clone https://github.com/afonsozango/PROJECTO-WEB.git
```

Isto cria uma pasta com o nome do repositório (ex: `PROJECTO-WEB`).

Se o código estiver noutro repositório com outro nome (ex: `edumatic-front`), usa **essa** URL no `git clone`.

Entra na pasta:

```
cd PROJECTO-WEB
```

(Substitui pelo nome real da pasta que apareceu.)

Ainda não tens Git / não tens link?

Podes copiar a pasta do projecto por **pen USB**, **ZIP** ou **Google Drive**. O importante é teres dentro ficheiros como `package.json`, `server.js`, `schema.sql`.

Passo 2 — Instalar dependências do Node (`npm install`)

Dentro da pasta do projecto:

```
npm install
```

- Demora 1–5 minutos na primeira vez.
- Cria a pasta `node_modules` (não apagues).
- Se der erro de rede, verifica internet e tenta de novo.

Passo 3 — Criar a base de dados (só na primeira vez)

Ainda na pasta do projecto.

3.1 — Criar a base `edumatic`

```
psql -U postgres -c "CREATE DATABASE edumatic;"
```

- Se disser que já existe, está OK — podes ignorar.

3.2 — Criar tabelas

```
psql -U postgres -d edumatic -f schema.sql
```

3.3 — Dados de teste (alunos fictícios + contas secretaria/director)

```
psql -U postgres -d edumatic -f seed.sql
```

Em cada comando, o terminal pode pedir a **senha do utilizador postgres**.

Windows: se `psql` não for reconhecido, abre “SQL Shell (psql)” do menu PostgreSQL ou adiciona PostgreSQL ao PATH.

Senha do postgres diferente de 1234 ?

Edita o ficheiro `src/config/db.js` e muda a linha `password: '1234'` para a tua senha real.

Passo 4 — Ligar o site (`npm start`)

```
npm start
```

Se estiver tudo bem, vês algo como:

```
Servidor a correr em http://localhost:3000
```

Não feches esta janela do terminal enquanto quiseres usar o site — ela é o “motor ligado”.

Abre o navegador e vai a:

<http://localhost:3000>

Passo 5 — Modo desenvolvimento (opcional)

Se vais **alterar código** e queres que o servidor reinicie sozinho:

```
npm run dev
```

(Requer `nodemon`, já listado no `package.json`.)

Todos os dias — como voltar a rodar

Não precisas de clonar nem de `npm install` outra vez (só se alguém adicionar pacotes novos).

1. Liga o **PostgreSQL** (se no teu PC ele não arranca sozinho).
2. Abre o terminal na **pasta do projecto**.
3. Corre:

```
npm start
```

4. Abre <http://localhost:3000>

Como parar o servidor

No terminal onde corre `npm start` :

- Carrega **Ctrl + C**
- Confirma com `S` ou Enter se perguntar

O site deixa de funcionar até voltares a correr `npm start` .

Resumo dos comandos (copiar e colar)

Ordem na primeira instalação:

```
cd ~/Desktop
git clone https://github.com/afonsonzango/PROJECTO-WEB.git
cd PROJECTO-WEB
npm install
psql -U postgres -c "CREATE DATABASE edumatric;"
psql -U postgres -d edumatric -f schema.sql
psql -U postgres -d edumatric -f seed.sql
npm start
```

Ordem no dia a dia:

```
cd PROJECTO-WEB
npm start
```

Configuração importante (ligação à base de dados)

Ficheiro: src/config/db.js

Campo	Valor por defeito no projecto
host	localhost
porto	5432
base	edumatric
utilizador	postgres
senha	1234

Se no teu PC a senha do postgres for outra, **muda só aqui** (ou alinha a senha do postgres com 1234).

Erros frequentes na instalação

Mensagem / problema	O que fazer
git: command not found	Instala o Git (Passo 0).
node: command not found	Instala o Node.js (Passo 0).
npm install falha	Verifica internet; apaga node_modules e corre npm install outra vez.
psql: command not found	Instala PostgreSQL ou usa o SQL Shell do Windows.
password authentication failed	Senha errada do postgres — corrige em src/config/db.js .
connection refused (PostgreSQL)	Serviço PostgreSQL não está ligado — inicia-o.
EADDRINUSE porta 3000	Já há outro npm start aberto — fecha o outro terminal ou mata o processo.
Site abre mas login falha	Correste seed.sql ? Utilizador secretaria / diretor ?
Pasta do clone com outro nome	Normal — entra na pasta que o git clone criou, não importa o nome.

Partilhar o projecto com um colega (resumo)

1. Envia-lhe o **link do GitHub** ou um **ZIP** da pasta (sem `node_modules` — é pesado; ele corre `npm install`).
2. Diz-lhe para ler **esta secção 7** deste ficheiro.
3. Diz-lhe a **senha do postgres** do PC dele (ou que use `1234` e configure igual).
4. Depois de `npm start`, o endereço é sempre <http://localhost:3000> (só no computador dele; não é um site na internet pública, a menos que alguém publique o servidor).

8. Como entrar no site (logins de teste)

Secretaria

- Página: <http://localhost:3000/login/secretaria>
- Utilizador: `secretaria`
- Senha: `123456` (ou a que estiver na base de dados)

Director

- Página: <http://localhost:3000/login/diretor>
- Utilizador: `diretor`
- Senha: `123456` (ou a que estiver na base de dados)

O ficheiro `seed.sql` original usa a senha `senha123`. Se alguém mudou na base de dados, usa a senha actual.

Aluno (só se já foi aprovado)

- Página: <http://localhost:3000/login/aluno>
- Precisa de: **ID oficial + email**

Exemplo de aluno de teste (depois do `seed.sql`):

- ID: `JOAB1001`
- Email: `joao.baptista@gmail.com`

9. Páginas principais do site

O que queres fazer	Onde ir
Página inicial	<code>/</code>
Pedir matrícula	<code>/fazer-matricula</code>
Ver se fui aceite	<code>/status-matricula</code>
Login aluno	<code>/login/aluno</code>
Painel aluno	<code>/aluno/painel</code>
Login secretaria	<code>/login/secretaria</code>
Painel secretaria	<code>/secretaria/painel</code>
Login director	<code>/login/diretor</code>
Painel director	<code>/diretor/painel</code>

10. ID oficial — o que é?

Quando a secretaria **aprova**, o sistema inventa um código único para o aluno.

Formato aproximado:

2 letras do primeiro nome + 2 letras do último nome + 4 números

Exemplo: **João Baptista** → algo como **JOAB1001**

Esse código é o “número de aluno” para login.

11. Ficheiros que o aluno envia

- Fotos e certificados vão para: `public/uploads/`
 - O nome do ficheiro é alterado para não haver dois ficheiros com o mesmo nome
 - Limite: cerca de **10 MB** por ficheiro
-

12. Problemas comuns (e o que verificar)

“Não consigo entrar como secretaria/director”

- O servidor está a correr? (`npm start`)
- Utilizador correcto? `secretaria` e `director` (não “director” com c em inglês)
- A senha na base de dados é a que estás a usar?
- PostgreSQL está ligado?

“O site não abre”

- Erro no terminal ao correr `npm start` ?
- Porta 3000 já ocupada por outro programa?

“Não aparecem alunos”

- Correste `seed.sql` ?
- PostgreSQL com a base `edumatic` criada?

“Aluno não faz login”

- Só alunos **Aprovados** têm ID oficial
 - Tem de usar o **ID + email** correctos
-

13. Resumo final (3 linhas)

1. **Aluno** pede matrícula → fica **Pendente**.
 2. **Secretaria** aprova ou recusa → se aprovar, nasce o **ID**.
 3. **Director** vê totais e decide se a escola **aceita novos pedidos** ou não.
-

14. Onde ler mais detalhes?

Documentação técnica completa: [README.md](#)